

235. Lowest Common Ancestor of a Binary Search Tree Medium

Acceptance: 69.9% | [LeetCode](#)

Given a binary search tree (BST), find the lowest common ancestor (LCA) node of two given nodes in the BST.

According to the [definition of LCA on Wikipedia](#): “The lowest common ancestor is defined between two nodes `p` and `q` as the lowest node in `T` that has both `p` and `q` as descendants (where we allow **a node to be a descendant of itself**).”

Example 1:

```
**Input:** root = [6,2,8,0,4,7,9,null,null,3,5], p = 2, q = 8
**Output:** 6
**Explanation:** The LCA of nodes 2 and 8 is 6.
```

Example 2:

```
**Input:** root = [6,2,8,0,4,7,9,null,null,3,5], p = 2, q = 4
```

```
**Output:** 2
**Explanation:** The LCA of nodes 2 and 4 is 2, since a node can be a descendant of itself according to the LCA definition.
```

Example 3:

```
**Input:** root = [2,1], p = 2, q = 1
**Output:** 2
```

Constraints:

- The number of nodes in the tree is in the range `[2, 105]`.
- `-109 <= Node.val <= 109`
- All `Node.val` are **unique**.
- `p != q`
- `p` and `q` will exist in the BST.

Claude Opus Solution

Lowest Common Ancestor of a BST

1. Intuition

The **key insight** is that we should exploit the **BST property**, not treat this as a generic binary tree problem.

In a BST: - All values in the **left subtree** are **less than** the node. - All values in the **right subtree** are **greater than** the node.

So for nodes `p` and `q`, the LCA is the **first node** (from the top) where `p` and `q` "split" into different subtrees — or where the current node equals one of them.

There are exactly 3 cases for any node we visit: 1. **Both `p` and `q` are greater than current node** → LCA must be in the right subtree. 2. **Both `p` and `q` are smaller than current node** → LCA must be in the left subtree. 3. **They diverge** (one is $\leq$ node, one is $\geq$ node, or node equals one of them) → **this node is the LCA** (the split point).

This lets us walk a single path from root to the split point — no full tree traversal needed.

2. Approach (Pseudocode)

```
function LCA(root, p, q):
    while root is not None:
        if p.val > root.val AND q.val > root.val:
            root = root.right      # both lie in right subtree
        else if p.val < root.val AND q.val < root.val:
            root = root.left       # both lie in left subtree
        else:
            return root            # split point found → this is LCA
```

The third condition handles "diverge" AND the case where `root` equals `p` or `q` (a node can be its own ancestor).

3. Python Solution

```
# Definition for a binary tree node.
class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val = val
        self.left = left
        self.right = right
```

```

class Solution:
    def lowestCommonAncestor(self, root: 'TreeNode',
                             p: 'TreeNode', q: 'TreeNode') -> 'TreeNode':
        # Walk down from the root following BST rules.
        # We never need to backtrack because the BST property
        # tells us exactly which direction the answer lies.
        while root:
            # Case 1: Both p and q are larger than current node.
            # The LCA must lie in the RIGHT subtree.
            if p.val > root.val and q.val > root.val:
                root = root.right

            # Case 2: Both p and q are smaller than current node.
            # The LCA must lie in the LEFT subtree.
            elif p.val < root.val and q.val < root.val:
                root = root.left

            # Case 3: p and q split here (one goes left, one goes right),
            # OR root IS p or q. This node is the lowest common
            # ancestor — return it immediately.
            else:
                return root

        # Per the constraints, p and q always exist, so this is
        # never reached. Returned only for completeness.
        return None

```

Cleaner variant using min/max (matches reference solution)

```

class Solution:
    def lowestCommonAncestor(self, root, p, q):
        # Precompute the boundary values once.
        low, high = min(p.val, q.val), max(p.val, q.val)
        while root:
            if root.val > high:          # node bigger than both → go left
                root = root.left
            elif root.val < low:         # node smaller than both → go right
                root = root.right
            else:                        # low <= root.val <= high → split point
                return root

```

4. Complexity Analysis

Metric	Complexity	Explanation
Time	<code>O(h)</code>	We descend exactly one path from root to the LCA. <code>h</code> is the tree height — <code>O(log n)</code> for a balanced BST, <code>O(n)</code> worst case (skewed tree).
Space	<code>O(1)</code>	The iterative version uses only a couple of variables — no recursion stack, no extra data structures.

A recursive version would have the same time but `O(h)` space due to the call stack.

5. Alternative Approaches

- Recursive BST walk** — Same logic as above but recursive: `python def lowestCommonAncestor(self, root, p, q): if p.val > root.val and q.val > root.val: return self.lowestCommonAncestor(root.right, p, q) if p.val < root.val and q.val < root.val: return self.lowestCommonAncestor(root.left, p, q) return root` Same `O(h)` time, but `O(h)` space from recursion.
- Generic Binary Tree LCA (LeetCode 236)** — Works on *any* binary tree by recursively searching both subtrees for `p` and `q`. This **ignores the BST property** and runs in `O(n)` time / `O(n)` space. It's correct but wastes the BST structure — use it only if you can't rely on the ordering.
- Find both paths, then compare** (the `my_solution.py` approach) — Record the root-to-node path for both `p` and `q`, then find the last common element. Works but uses `O(h)` extra space for storing paths and does two traversals. The single-walk approach is strictly better.

Takeaway pattern: Whenever a problem mentions a **BST**, ask "How can the ordering property turn a full search into a single guided walk down one path?" — that's the recurring theme that turns $O(n)$ solutions into $O(h)$.

Codex Solution

1. Intuition

In a **Binary Search Tree (BST)**:

- All values in the left subtree are smaller than the current node.
- All values in the right subtree are larger than the current node.

For two nodes `p` and `q`, their **lowest common ancestor** is the first node where their search paths split.

At any node `root`:

- If both `p` and `q` are smaller than `root`, then the LCA must be in the left subtree.
- If both `p` and `q` are larger than `root`, then the LCA must be in the right subtree.
- Otherwise, `root` is the split point, meaning one node is on one side and the other is on the other side.
- Or, `root` is equal to either `p` or `q`.
- In both cases, `root` is the LCA.

The key pattern is using the **BST ordering property** to avoid searching the entire tree.

2. Approach

Pseudocode

```
function lowestCommonAncestor(root, p, q):  
  
    smaller = min(p.val, q.val)  
    larger = max(p.val, q.val)  
  
    current = root  
  
    while current is not null:  
  
        if current.val < smaller:  
            # Both p and q are larger than current  
            # So LCA must be in the right subtree  
            current = current.right  
  
        else if current.val > larger:  
            # Both p and q are smaller than current  
            # So LCA must be in the left subtree  
            current = current.left  
  
        else:  
            # current.val is between p.val and q.val  
            # This is the split point, so current is LCA  
            return current
```

3. Python Solution

```
# Definition for a binary tree node.  
# class TreeNode:  
#     def __init__(self, x):  
#         self.val = x  
#         self.left = None  
#         self.right = None
```

```
class Solution:  
    def lowestCommonAncestor(  
        self,  
        root: 'TreeNode',  
        p: 'TreeNode',
```

```

    q: 'TreeNode'
) -> 'TreeNode':
    # Store the smaller and larger values between p and q.
    # This makes comparisons cleaner.
    low = min(p.val, q.val)
    high = max(p.val, q.val)

    # Start searching from the root of the BST.
    current = root

    while current:
        # Case 1:
        # If current value is smaller than both p and q,
        # then both nodes must be in the right subtree.
        if current.val < low:
            current = current.right

        # Case 2:
        # If current value is larger than both p and q,
        # then both nodes must be in the left subtree.
        elif current.val > high:
            current = current.left

        # Case 3:
        # current.val lies between p.val and q.val.
        #
        # This means:
        # - p and q are on different sides of current, or
        # - current is equal to p or q.
        #
        # In either case, current is the lowest common ancestor.
        else:
            return current

```

4. Complexity Analysis

Let h be the height of the BST.

Time Complexity: $O(h)$

We move down the tree one level at a time.

- In a balanced BST, $h = \log n$, so time complexity is $O(\log n)$.
- In a skewed BST, $h = n$, so time complexity is $O(n)$.

Therefore, the general time complexity is:

$O(h)$

Space Complexity: $O(1)$

The iterative solution only uses a few variables:

- low
- high
- current

No recursion stack or extra data structures are used.

Therefore, space complexity is:

$O(1)$

5. Alternative Approaches

Alternative 1: Recursive BST Search

We can write the same logic recursively.

```

class Solution:
    def lowestCommonAncestor(
        self,

```

```

    root: 'TreeNode',
    p: 'TreeNode',
    q: 'TreeNode'
) -> 'TreeNode':

    # If both p and q are smaller, search left subtree.
    if p.val < root.val and q.val < root.val:
        return self.lowestCommonAncestor(root.left, p, q)

    # If both p and q are larger, search right subtree.
    if p.val > root.val and q.val > root.val:
        return self.lowestCommonAncestor(root.right, p, q)

    # Otherwise, root is the split point.
    return root

```

Complexity

- Time: $O(h)$
- Space: $O(h)$ due to recursion stack

Alternative 2: General Binary Tree LCA

We could ignore the BST property and solve it like a normal binary tree LCA problem. That approach checks both left and right subtrees recursively.

Complexity

- Time: $O(n)$
- Space: $O(h)$

However, this is less efficient because it does not use the BST ordering property.